Tenaye Info — Team Collaboration & GitHub Workflow

A guide for a 4-person team: who owns what, ready-to-use GitHub issues, and the exact flow from issue to merge.

1. Splitting the Work — Feature Ownership

The project's architecture already separates cleanly into feature slices, matching the folder structure. Each person owns one slice end-to-end — backend AND frontend for their feature — rather than splitting purely by "backend person" / "frontend person", since that creates more merge conflicts and blocking dependencies. As project manager, you (Person A) already completed Auth, Medicine Search, and Profile (Days 1-4) — the three issues below are what you'll create and assign to your three teammates.

Person	Owns	Backend files	Frontend files
You (PM)	Auth & User Profile — already done	models/User.js, authController.js, userController.js, authRoutes.js, userRoutes.js, authMiddleware.js	login.html, register.html, profile.html + matching js/pages, utils/auth.js
Person B	Medicine Search & OCR	models/Medicine.js, medicineController.js, ocrController.js, medicineRoutes.js, ocrRoutes.js, ocrService.js, drugLookupService.js	medicine-search.html, medicine-details.html + matching js/pages
Person C	Chatbot (Gemini)	models/ChatHistory.js, chatbotController.js, chatbotRoutes.js, chatbotService.js, config/gemini.js	chatbot.html + js/pages/chatbot.js, js/components/chatBubble.js
Person D	Pharmacy Locator, i18n, Polish	pharmacyController.js, pharmacyRoutes.js, pharmacyService.js	pharmacy-locator.html + js/pages, js/i18n.js, locale JSON files, navbar.js, footer.js, languageSwitcher.js

Shared files everyone touches (coordinate before editing): app.js, server.js, package.json, and any file under config/ or middleware/ used by more than one feature. Whoever needs to edit these should message the group first.

2. Issues to Create — One Per Teammate

As PM, create these three issues on GitHub (Issues tab → New Issue). Copy the title into the issue title field, and copy the rest into the description box. Assign each to the matching teammate and add it to a "Day 5-7" milestone or project board column if you're using one.

Issue for Person B

Title: Medicine photo upload — OCR identification + search polish

Description

Add photo-based medicine identification using Google Vision OCR, and finish out the medicine search frontend pages. Users should be able to either type a medicine name or upload a photo of a medicine package and get matched results.

Tasks

- Backend: config/googleVision.js and config/cloudinary.js — set up API clients using keys from .env
- Backend: services/ocrService.js — send uploaded image to Google Vision, extract text
- Backend: services/drugLookupService.js — take extracted text, search it against the Medicine collection (reuse the same $or/regex pattern from medicineController.js)
- Backend: controllers/ocrController.js + routes/ocrRoutes.js — POST /api/ocr/identify, accepts an image upload (multer), returns matched medicine(s)
- Frontend: pages/medicine-search.html + js/pages/medicineSearch.js — search bar UI, calls the existing search endpoint, renders results with js/components/medicineCard.js
- Frontend: pages/medicine-details.html + js/pages/medicineDetails.js — photo upload form, calls the new OCR endpoint, shows matched medicine or a "no match found" message

Acceptance criteria

- Typing a medicine name and searching returns real results from the database
- Uploading a clear photo of a medicine label returns a reasonable match, or a graceful "not found" message if it can't match
- No crash on invalid/blurry uploads — a real user-facing error message instead

Notes

OCR is the highest-risk piece of the whole project — if stuck on image preprocessing (blur, glare, rotation), flag it in the group chat early rather than late. Search-by-name should be finished and solid first, since it's the fallback if OCR isn't ready in time.

Issue for Person C

Title: AI chatbot — Gemini integration (login-only feature)

Description

Build the AI health chatbot using the Gemini API, grounded in our own medicine database so it doesn't invent dosage or safety information. Per the README, this feature is only available to logged-in users — logged-out users clicking it should be redirected to register.

Tasks

- Backend: config/gemini.js — set up the Gemini client using GEMINI_API_KEY from .env
- Backend: models/ChatHistory.js — schema storing userId, role (user/assistant), message, timestamps
- Backend: services/chatbotService.js — look up relevant medicines from our own DB based on the question, build a grounded prompt, call Gemini, return the reply
- Backend: controllers/chatbotController.js + routes/chatbotRoutes.js — POST /api/chatbot/message, protected by authMiddleware (protect)
- Frontend: pages/chatbot.html + js/pages/chatbot.js — chat UI, message input, calls the chatbot endpoint
- Frontend: js/components/chatBubble.js — reusable function to render one message bubble (user vs assistant styling)
- Frontend: redirect-to-register behavior for logged-out users trying to access the chatbot (reuse the isLoggedIn() pattern from profile.js)

Acceptance criteria

- Logged-out users clicking "Talk with Tenaye AI" get redirected to register, not an error
- Logged-in users can send a message and get a real Gemini-generated reply
- Asking about a medicine that exists in our database produces an answer that reflects our data, not generic/invented info
- Chat history is saved per-user in MongoDB

Notes

Use model "gemini-3.6-flash" (current stable model as of this project — Google updates model names periodically, so check the Gemini API docs if you get a 404 model-not-found error). Coordinate with Person B on how drugLookupService.js works, since your context-building logic will query the same Medicine data.

Issue for Person D

Title: Pharmacy locator + multilingual UI polish

Description

Build the pharmacy locator feature (find nearby pharmacies using the user's location), and take ownership of the shared UI pieces — navbar, footer, language switcher — plus final polish and consistency across all pages.

Tasks

- Backend: services/pharmacyService.js — call Google Maps/Places API (or OpenStreetMap if avoiding API keys) with user coordinates, return nearby pharmacies
- Backend: controllers/pharmacyController.js + routes/pharmacyRoutes.js — GET /api/pharmacies/nearby?lat=&lng=
- Frontend: pages/pharmacy-locator.html + js/pages/pharmacyLocator.js — request browser geolocation permission, call the endpoint, render results with js/components/pharmacyListItem.js
- Frontend: js/components/navbar.js and footer.js — shared across every page, used consistently instead of each page having its own copy-pasted nav markup
- Frontend: finish js/i18n.js integration and locale JSON files (en/am) across ALL pages built by every teammate, not just your own — this is a cross-cutting task
- Final pass: consistent styling (Tailwind classes) across every page, working links between pages, the medical disclaimer text from the README visible somewhere appropriate

Acceptance criteria

- Clicking "Find Pharmacy" prompts for location permission and shows real nearby results (or a clear message if location is denied)
- Every page uses the same navbar/footer component, not duplicated markup
- Language switcher works correctly on every page, not just the ones it was originally tested on
- No broken links between pages; consistent look and feel across the whole app

Notes

This issue depends on the other three being mostly done, since polish means touching everyone's pages — plan to start the pharmacy locator backend/frontend early (it's independent), and do the cross-page polish pass last, closer to the end of the week.

3. One-Time Repo Setup (You Do This First)

From inside the project's root folder:

```
git init
git add .
git commit -m "Initial commit: Days 1-4 complete"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/tenaye-info.git
git push -u origin main
```

Then on GitHub → repo Settings → Collaborators → add your three teammates by GitHub username so they have push access. Each teammate then clones it once:

```
git clone https://github.com/YOUR_USERNAME/tenaye-info.git
cd tenaye-info
```

4. Branching Strategy

- **main** — always working, deployable code. Nobody commits to main directly.
- **Feature branches** — one branch per issue, named clearly: feature/ocr-search, feature/chatbot, feature/pharmacy-locator
- Each person works on their own branch, pushes it, opens a Pull Request into main, gets it reviewed, then merges.

5. The Workflow — Issue to Merge

Step 1 — You create the issue, assign the teammate

Use the three issue templates in Section 2 above, copy-pasted directly into GitHub's New Issue form.

Step 2 — They branch off an up-to-date main

```
git checkout main
git pull origin main
git checkout -b feature/chatbot
```

Step 3 — Work, committing in small chunks

```
git add src/controllers/chatbotController.js
git commit -m "Add chatbot controller with Gemini integration"
```

Step 4 — Push the branch

```
git push -u origin feature/chatbot
```

Step 5 — Open a Pull Request

1. On GitHub, click "Compare & pull request" on the pushed branch
2. Title it clearly, e.g. "Chatbot: Gemini integration"
3. In the description, write "Closes #3" (using the actual issue number) so the issue auto-closes on merge
4. Request a review from you (or another teammate) before merging

Step 6 — Review & Merge

A reviewer checks the diff on GitHub — does it run, does it follow the same pattern as existing code. Approve, then "Squash and merge" to keep main's history clean. Delete the feature branch afterward.

Step 7 — Everyone syncs

```
git checkout main
git pull origin main
```

6. Handling Merge Conflicts

1. Git marks conflicting sections with <<<<<<<, =======, >>>>>>> markers in the file
2. Manually edit to keep the correct combined version, delete the marker lines
3. git add the fixed file, then git commit to complete the merge

Since each person owns mostly separate files, conflicts should be rare. The main shared-risk files are app.js and package.json — pull right before editing those, push right after.

7. Suggested Daily Rhythm

- **Morning (10 min):** quick sync — what's done, what's today, any blockers
- **During the day:** work on your own branch, commit and push often
- **End of day (15 min):** push, open PRs for anything finished, review and merge before the day ends — don't let PRs pile up